Otter Delivery

AI-Assisted Food Discovery and Delivery with Microservices

Group: Otter Delivery
Technische Universität Berlin — PP Microservices 2026

Authors: Minjian Li · Nikolay Iliev · Sohrab Dokmehchin

01 Product Idea

AI-assisted food discovery for international and multicultural users.

Key idea: AI recommends restaurants and dishes. Users stay in control and create the final order manually.

Why this matters

- Unfamiliar menus and dish names
- Dietary uncertainty for international users
- Decision fatigue caused by too many options
- Need for discovery, not only keyword search

02 Service Summary

Translation Service
Menu and dish translation for multilingual user support.
Technologies: spring, PostgreSQL, supabase

Restaurant Service
Restaurant and menu management, search and availability tracking.
Technologies: spring, PostgreSQL, supabase

Recommendation Service
Restaurant and dish recommendations based on preferences and order history.
Technologies: Gemini, PostgreSQL, supabase

Order Service
Order creation, checkout integration, tracking and status management.
Technologies: spring, PostgreSQL, supabase

Driver Service
Driver assignment, position updates and simplified delivery ETA.
Technologies: FastAPI, PostgreSQL, supabase

Profile Service
Customer profiles, preferences, addresses and order history aggregation.
Technologies: spring, PostgreSQL, supabase

03 Microservice Architecture

DOCKER Containerized Services

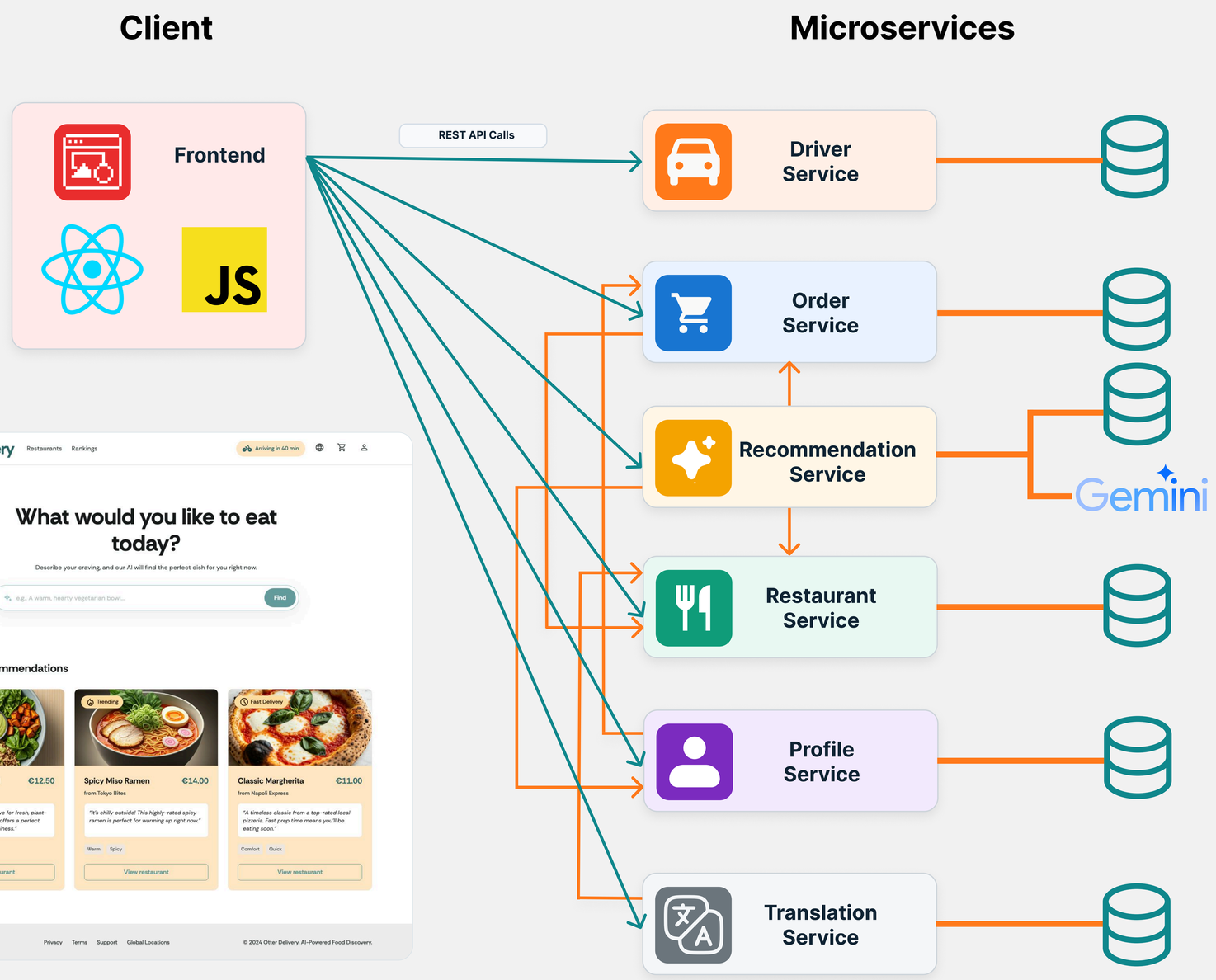
DEP Independent Deployment

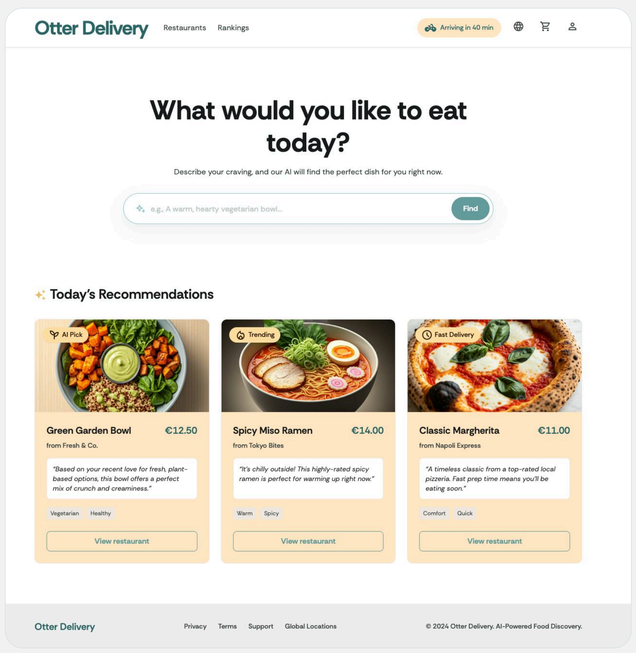
CI/CD Automated Build & Deployment

API REST Communication

DB Database per Service

TECH Technology Heterogeneity





04 Design Choices

- FastAPI for Driver Service: FastAPI enabled rapid development of lightweight REST APIs with high performance and seamless integration with Python-based components.
- Recommendation Criteria: Scores combine preferences, query matches, restaurant data (e.g. rating) and previous orders.
- Translation Service is cached to reduce token usage and API costs.
- Each backend service communicates through REST APIs for simplicity.
- Database per Service: Each service owns its data and persists it independently in PostgreSQL/Supabase.
- No API Gateway: Direct frontend-to-service communication keeps the MVP simpler and easier to debug.
- Technology per Service: We chose technologies based on service needs: Spring Boot for stable business services and FastAPI for AI/driver-related services

05 Problems and Decisions

Problem Frontend mock data did not match real backend responses.
Decision **Added adapter / UI view-model layer.**

Problem Supabase connection failed because of IPv4/IPv6 incompatibilities.
Decision **Use Supabase Session Pooler and PostgreSQL SSL configuration.**

Problem Recommendations depend on external services (Restaurant Service, Order Service, Gemini), which may become unavailable.
Decision **Implemented fallback mechanisms (rule-based scoring, mock data, in-memory storage) to keep the system operational.**

Problem Managing multiple service endpoints without a central gateway.
Decision **For the MVP, we kept direct REST calls to reduce complexity and documented service endpoints clearly.**